Semester Access Control and RLS Standard

Document ID: SEC-AC-003 | Version: 0.1 draft | Issued: 2026-10-06
Owner: Database/security lead (appoint) | Status: Proposed; verify schema before use

1. Policy

Deny by default. Enforce tenant, subject, capability, resource, and operation boundaries on trusted backend paths and database objects. Derive subject identity from a validated authentication context, not a request-supplied user ID. Tenant IDs supplied by clients are selectors, not proof of membership. Admin means explicitly scoped authority, not universal access. Policy service failure denies sensitive actions.

Working permission model: app_roles, role_capabilities, role_grants, private.has_capability. These names come from prior project context, not current schema verification. Do not invent the helper signature or copy unverified SQL into production. Inspect migrations, ACLs, ownership, token behavior, and the actual helper implementation first.

2. Role and capability governance

Define capability as action + resource + scope, with explicit tenant, optional expiry, approval, and audit trail. Separate platform operators, institutional administrators, faculty/advisors, students, integrations, and background workers. Role creation/assignment/revocation is privileged. Users cannot assign themselves roles or update authorization metadata. Untrusted user-editable metadata cannot establish authority. Evaluate verified grants, scope, expiry, suspension, and revocation at enforcement time.

High-risk grants require approver independence where feasible and stronger authentication. Break-glass is time-bounded, reason-coded, monitored, and independently reviewed. Service accounts have narrow purposes, owners, secret rotation, and no interactive/shared login.

3. Database grants and exposure

Grants control access to objects; RLS controls rows. Review both. Catalog API-exposed schemas, tables, views, materialized views, functions, and sequences. Enable RLS for exposed tables that hold restricted rows. A table with RLS but no policies denies normal-role row access; that can be intentional, not automatically a vulnerability. Record the intent and test application availability. Privileged bypass roles and owners require separate controls.

Public read exceptions require data classification, minimal columns, rationale, reviewer, and evidence. Views require deliberate ownership/security semantics: test whether underlying RLS applies for the actual caller and database version. Do not assume RLS protects every view or RPC. Review partition and direct-table access paths. Never expose private permission helpers as public RPCs without explicit need.

4. Policy requirements

SELECT: enforce tenant membership and record visibility.
INSERT: WITH CHECK must validate tenant/owner and capability; prevent identity spoofing.
UPDATE: USING restricts existing rows, WITH CHECK restricts resulting rows; additionally prevent unauthorized changes to protected columns through grants, carefully designed triggers, or approved RPCs.
DELETE: require deletion authority, tenant scope, and retention/legal-hold handling.

An UPDATE policy alone is not a complete read policy. Test policy combinations, including permissive OR composition and restrictive policies. Bind helper results to the current subject and target tenant. Index predicates where appropriate and profile with realistic data without weakening enforcement.

5. SECURITY DEFINER and RPC requirements

Prefer SECURITY INVOKER. Definer functions require necessity, least-privilege owner, narrow execution grants, fixed safe search_path (prefer empty with schema-qualified references), authorization inside the function, input bounds, output minimization, and negative caller tests. Pinning search_path does not grant authorization or restrict callers. Inspect both PUBLIC inheritance and explicit role grants. Revoke unintended EXECUTE from PUBLIC, anon, authenticated, and other directly granted roles as appropriate; regrant only approved callers. Review default privileges for the actual creator roles so future functions do not reopen access.

RLS does not constrain functions as an authorization mechanism by itself. A definer function owned by a bypass-capable role can read or write otherwise protected rows. Validate ownership, caller subject, tenant membership, and action for each resource. Avoid caller-controlled identifiers as authorization proof and uncontrolled dynamic SQL. Review overloaded signatures, triggers, recursive permission checks, side effects, and information leaks. Tests must cover direct RPC invocation independent of the UI.

6. Illustrative SQL only

The following applies to a HYPOTHETICAL function. It is not a migration for Semester; replace the exact schema/name/signature after discovery and review.

```
BEGIN;
ALTER FUNCTION private.example_authorized_lookup(uuid) SET search_path = '';
REVOKE ALL ON FUNCTION private.example_authorized_lookup(uuid) FROM PUBLIC;
REVOKE ALL ON FUNCTION private.example_authorized_lookup(uuid) FROM anon;
REVOKE ALL ON FUNCTION private.example_authorized_lookup(uuid) FROM authenticated;
-- Grant only after confirming internal authorization and required calling role.
GRANT EXECUTE ON FUNCTION private.example_authorized_lookup(uuid) TO authenticated;
COMMIT;
```

This example does not modify the function body, prove its owner is safe, or protect against other granted roles. Audit all three separately. Schema-qualify every relation/reference in the implementation; review dynamic SQL independently.

7. Privileged backend and client separation

Never put secret/service-role keys in VITE_* variables, browser bundles, URLs, public logs, client storage, or preview artifacts. The public publishable/anon key is not a privileged secret; safety depends on correct grants/RLS. Backend use of service role bypasses database RLS, so authorize the user and tenant before every operation, minimize scope, and audit the result. Prefer caller-scoped clients for user requests where practical. Worker authority is validated independently from arbitrary payloads.

8. Storage, realtime, and derived data

Private buckets require object policies covering read, upload, overwrite, delete, and movement. Validate tenant/owner at object metadata and storage path boundaries; a guessed path must not confer access. Signed URLs are bearer access until expiry; keep lifetime bounded and document revocation limitations. Test whether previously issued URLs remain usable after membership revocation.

Authorize realtime channels/subscriptions and verify actual deployed behavior on revocation. Search, analytics, exports, caches, logs, notifications, and attachments must not leak data denied at the source. Cache keys require tenant/subject scope as applicable. User-facing data must record source/freshness; stale authorization must not restore revoked access.

9. Required test matrix

Fixtures: anonymous; student A and student B in tenant T1; student C in tenant T2; scoped staff; institution admin T1; suspended user; revoked member; expired grant; integration worker; privileged backend. Execute against actual API/database roles, not only a postgres-owner connection.

AC-01: anonymous reads/writes protected objects -> denied/no protected rows.
AC-02: A reads B's private resource -> denied; no sensitive error detail.
AC-03: T1 member reads/mutates T2 resource -> denied on API, RPC, storage, exports, and realtime.
AC-04: insert spoofed owner or tenant -> rejected; no row created.
AC-05: update existing row to another tenant/owner -> rejected; original unchanged.
AC-06: self-grant role/capability -> rejected; no grant/event created.
AC-07: direct definer RPC with foreign ID -> denied; no leaked row/side effect.
AC-08: revoked/expired/suspended subject reuses old session -> sensitive access denied according to documented revocation bound.
AC-09: mixed permissive policies -> no unexpected allow.
AC-10: public view/derived export -> no unauthorized underlying rows/columns.
AC-11: privileged API receives unauthorized request -> denied BEFORE service-role operation.
AC-12: guessed object path and stale signed URL -> behavior matches approved policy and expiry limits.
AC-13: forged user metadata/claims, expired token, wrong issuer/audience -> denied.
AC-14: concurrent revoke/write and repeated/out-of-order workflow event -> no restoration of authority; consistent recorded outcome.
AC-15: positive owner and scoped staff cases -> authorized operations succeed without

overbroad results.
AC-16: fresh database plus upgraded database -> identical intended grants/policy behavior.

For every case collect actor/tenant, fixture IDs, endpoint/SQL, operation, expected status/rows, actual status/rows, side-effect check, commit, environment, timestamp, and sanitized artifact. Denied SELECT may return zero rows rather than an HTTP error; assert absence of forbidden data. Transaction rollback fixtures where possible and clean up test data safely.

10. Migration and drift controls

Inventory pg_class RLS flags, pg_policies, relation/schema/function ACLs, pg_proc ownership/prosecdef/proconfig, and API settings. Compare baseline to intended configuration. Catalogs are discovery evidence, not sufficient proof of enforcement. Test fresh and existing database paths; review default privileges for creator roles. Re-run after migrations, role changes, API schema changes, and privilege changes. Block unapproved exposure.

11. Incident handling

If unauthorized access is plausible: disable the narrow affected path or revoke the unsafe grant, preserve logs and deployment/migration references, assess scope, notify incident command, implement a reviewed fix, add regression tests, and retest all equivalent paths. Do not blindly disable all database access without evaluating recovery/availability consequences.
Privacy/legal evaluates notice obligations.

Sources

https://supabase.com/docs/guides/database/functions
https://supabase.com/docs/guides/database/postgres/row-level-security
https://supabase.com/docs/guides/api/securing-your-api